

DARKO — Kosmi-Inspired Virtual Hangout Platform

Product Requirements Document + Master Development Prompt

Version: 1.0

Date: 2026-08-18

Budget target: █0 / free-tier-first

Product goal: Build a premium, original web platform inspired by the functional category of Kosmi, without copying its branding, proprietary assets, source code, or exact visual design.

1. Product Vision

DARKO is a browser-first virtual hangout platform where people can create a room and instantly invite friends to watch content together, talk through voice/video, share screens, chat, exchange files/links, play multiplayer games, browse together, and customize their shared space.

The product must feel like a premium social entertainment operating system rather than a basic watch-party page.

Core principle:

Create a room, share one link, and everything needed for a digital hangout is inside the room.

Guests should be able to join without creating an account. Google authentication is available for users who want persistent profiles, saved rooms, preferences, history, or ownership.

2. Research-Based Feature Baseline

Current Kosmi materials describe a browser-based entertainment/hangout product with rooms, synchronized video, games, shared browsing, screen sharing, link/file sharing, voice/video/messaging, customizable rooms, and public/private room concepts. Kosmi also promotes no-download access and free usage.

DARKO should reproduce the underlying product category and workflows while using:

- original branding
- original UI
- original copy
- original assets
- original architecture
- open-source/free technologies where possible

Do not scrape, proxy, or redistribute copyrighted streaming content.

3. Goals

Primary goals

1. Zero-cost-first architecture.
2. Guest-first room joining.
3. Google login as optional authentication.
4. Real-time multiplayer room experience.
5. P2P WebRTC wherever practical.
6. Maximum functionality possible without paid APIs.
7. Premium, unique UI.
8. Excellent desktop/tablet/mobile responsiveness.
9. Modular architecture so features can be added later.
10. Strong privacy and abuse controls.

Secondary goals

- PWA installability.
- SEO landing pages.
- Shareable room URLs.
- Room themes.
- Public discovery.
- Multiplayer games.
- File/link sharing.
- Watch queues/playlists.
- Reactions.
- Presence and typing indicators.

Non-goals

- Do not bypass DRM.
- Do not download/re-host Netflix, Prime Video, Disney+, HBO, or other protected content.
- Do not implement copyright-infringing content libraries.
- Do not pretend a paid service is free.
- Do not depend on proprietary APIs when a browser-native/open alternative exists.

4. Users

Guest

Can:

- enter display name
- join room by link
- use permitted room features
- chat
- voice/video
- screen share
- watch supported content
- play games

Authenticated user

Can additionally:

- maintain profile
- create/manage persistent rooms
- save preferences
- save room templates
- view recent rooms
- manage owned rooms
- use Google account identity

Host

Can:

- change room settings
- control media synchronization
- control permissions
- mute/kick participants
- lock room
- enable/disable activities
- transfer host role

Moderator

Can:

- moderate messages
- mute/kick
- manage room permissions
- manage public room visibility

Admin

Can:

- moderate platform-wide content
- inspect abuse reports
- manage public rooms
- view system health
- manage feature flags
- manage bans

5. Main User Journey

Landing page

- Create Room
- choose room type/theme
- enter display name or Google login
- room created
- invite link copied
- friends join
- permissions/camera/microphone prompt
- room lobby
- shared activity selected
- watch/play/talk
- switch activities without leaving room

Join flow:

Room URL

- display name / Google login
- optional device permissions
- room
- synchronized state received
- participant presence established

6. Information Architecture

Public

- `/`
- `/features`
- `/games`

- `/watch`
- `/about`
- `/privacy`
- `/terms`
- `/safety`
- `/login`
- `/signup`
- `/join/[roomId]`

Authenticated

- `/dashboard`
- `/rooms`
- `/rooms/create`
- `/settings`
- `/profile`

Room

- `/room/[roomId]`

Admin

- `/admin`
- `/admin/reports`
- `/admin/users`
- `/admin/rooms`
- `/admin/system`

7. Landing Page

Hero:

- DARKO logo
- strong headline
- short value proposition
- Create Room CTA
- Join Room CTA
- animated premium virtual-room visual

Sections:

1. Watch together

2. Talk together
3. Play together
4. Share anything
5. Customize your room
6. Guest-first access
7. Privacy/security
8. PWA/cross-device
9. final CTA

Do not copy Kosmi's wording or layout.

8. Room Experience

The room is the heart of the application.

Desktop layout:

- top bar
- activity/workspace area
- participant strip/sidebar
- chat panel
- bottom media/call controls
- optional right-side activity drawer

Mobile:

- activity occupies main area
- bottom navigation
- expandable participants
- slide-over chat
- compact call controls
- touch-friendly controls

Room modes:

- Watch
- Screen Share
- Browse
- Games
- Files
- Whiteboard
- Chat Lounge

9. Watch Party

Supported free-first sources

Priority:

1. YouTube
2. direct public video URLs
3. user-selected local video via browser/P2P where technically feasible
4. HLS where browser-compatible and legally accessible
5. Twitch/public embeddable sources where permitted
6. other embeddable/public sources

For services protected by DRM:

- do not attempt DRM bypass
- provide screen-sharing workflow
- explain that every participant may need their own subscription
- never proxy protected streams through DARKO

Synchronization

Room media state:

- source
- currentTime
- paused
- playbackRate
- volume policy
- updatedAt
- host/client authority
- media session/version

Events:

- PLAY
- PAUSE
- SEEK
- RATE_CHANGE
- SOURCE_CHANGE
- SYNC_REQUEST
- SYNC_RESPONSE
- PLAYLIST_ADD
- PLAYLIST_REMOVE

- PLAYLIST_REORDER

Use timestamps and server/client clock offset to reduce drift.

Clients periodically correct drift.

Do not send playback position every animation frame.

10. Playlist

Features:

- add URL
- add video
- reorder
- remove
- next
- previous
- autoplay next
- host-only or collaborative queue
- queue persistence for authenticated room owners

11. Voice and Video

Use WebRTC.

Features:

- microphone toggle
- camera toggle
- device selection
- speaker selection where browser permits
- screen share
- screen-share audio where browser permits
- participant tiles
- active speaker
- mute indicators
- connection quality indicator
- reconnect state

Architecture:

- Socket/WebSocket signaling

- STUN for NAT traversal
- optional TURN fallback
- mesh for very small rooms
- architecture must allow future SFU migration

Important:

A pure P2P mesh does not scale efficiently to large groups. The implementation must expose a media-provider abstraction so an SFU can be added later without rewriting the room layer.

12. Screen Sharing

Use browser-native `getDisplayMedia`.

Modes:

- screen
- window
- browser tab

Show:

- presenter badge
- stop-sharing control
- active presenter
- audio-share status

Never attempt to capture protected content without explicit browser/user permission.

13. Chat

Features:

- realtime messages
- emoji
- reactions
- reply
- mentions
- typing indicator
- read/received state where practical
- message timestamps
- delete own message
- moderator deletion

- rate limiting
- spam protection
- basic link preview
- image/file attachment using safe limits

Chat should continue while switching activities.

14. Presence

Track:

- joined
- left
- reconnecting
- audio enabled
- video enabled
- screen sharing
- current activity
- host/moderator role
- idle state

Use heartbeat + timeout.

Do not persist unnecessary presence history.

15. Room Permissions

Settings:

- public/private
- password
- locked/unlocked
- guest allowed
- chat allowed
- voice allowed
- video allowed
- screen share allowed
- file sharing allowed
- games allowed
- who can control media

- who can invite
- who can kick/mute

Roles:

- owner
- host
- moderator
- member
- guest

16. Public Rooms

Optional lobby:

- currently active public rooms
- activity type
- participant count
- room title
- language/category
- room thumbnail

Safety:

- report room
- report user
- hide room
- moderation
- automatic expiration
- rate limits

Do not expose private room metadata.

17. Room Customization

Premium original visual system.

Themes:

- Midnight Lounge
- Neon Arcade
- Cozy Cinema
- Cyber Loft

- Space Station
- Sunset Drive-In
- Minimal Studio

Customization:

- background
- accent
- ambient animation
- room icon
- room title
- wallpaper
- layout density
- glass/solid panels

Animations must respect `prefers-reduced-motion`.

18. Games

Free/open-source-first games.

MVP:

- Tic-Tac-Toe
- Connect Four
- Chess
- drawing/whiteboard mini-game
- reaction game

Phase 2:

- card games
- multiplayer board games
- additional open-source games

For games:

- authoritative room state
- deterministic event model where practical
- anti-cheat validation
- reconnect state recovery

Do not embed copyrighted ROMs or proprietary game assets.

19. Whiteboard

Features:

- pen
- highlighter
- shapes
- text
- eraser
- undo/redo
- clear
- cursor/presence
- collaborative realtime state

Use a compact operation/event model rather than broadcasting full canvas snapshots constantly.

20. File Sharing

Free-first approach:

- direct P2P transfer using WebRTC DataChannels
- local browser file selection
- transfer progress
- cancel
- size limits
- file-type warnings
- no permanent server storage by default

Fallback:

- optional free-tier object storage adapter

Never make user files public by default.

21. Shared Links

Users can send links into room chat.

Features:

- URL validation
- safe preview
- title/favicon retrieval through safe server-side metadata service where available

- no arbitrary server-side URL fetching that can create SSRF vulnerabilities

22. Shared Browser

Important constraint:

A normal browser cannot simply render and control every arbitrary third-party website inside a cross-origin iframe.

Therefore implement:

1. link sharing
2. screen sharing
3. supported embeds
4. optional future remote-browser provider abstraction

Do not create an insecure open proxy.

23. Authentication

Google login:

- OAuth/OIDC provider
- secure callback
- account linking
- session management

Guest:

- generated anonymous session ID
- display name
- temporary avatar
- room-scoped identity

Guest accounts should not require persistent personal information.

24. Database

Prefer Supabase/Postgres free tier or another free-tier-compatible Postgres provider.

Suggested tables:

users

profiles

rooms
room_members
room_roles
room_settings
room_sessions
messages
message_reactions
playlists
playlist_items
room_events
reports
bans
saved_rooms
user_preferences

Do not persist high-frequency WebRTC signaling events permanently.

25. Realtime Architecture

Recommended:

Frontend:

Next.js + TypeScript

Backend:

Node.js + TypeScript

Realtime:

Socket.IO or native WebSocket

Media:

WebRTC

Database:

Postgres/Supabase

Auth:

Google OAuth + guest sessions

State:

Zustand or equivalent lightweight client store

Validation:

Zod

Styling:

Tailwind CSS

Animation:

Framer Motion

Icons:

Lucide

PWA:

manifest + service worker

26. Room Event Protocol

Use typed events.

Examples:

ROOM_JOIN

ROOM_LEAVE

PRESENCE_UPDATE

ROLE_UPDATE

ROOM_SETTINGS_UPDATE

CHAT_MESSAGE

CHAT_DELETE

CHAT_REACTION

TYPING_START

TYPING_STOP

MEDIA_PLAY

MEDIA_PAUSE

MEDIA_SEEK

MEDIA_SYNC

MEDIA_SOURCE_CHANGE

WEBRTC_OFFER

WEBRTC_ANSWER

WEBRTC_ICE

SCREEN_SHARE_START

SCREEN_SHARE_STOP

GAME_STATE

GAME_ACTION

WHITEBOARD_OPERATION

FILE_OFFER

FILE_ACCEPT

FILE_PROGRESS

FILE_COMPLETE

FILE_CANCEL

Every event must have:

- event ID
- room ID
- sender ID
- timestamp
- schema version where appropriate
- payload

Validate all incoming events.

27. Free Infrastructure Strategy

Target ■0.

Frontend:

- Cloudflare Pages or equivalent free static/Next-compatible deployment
- Vercel free tier as alternative where compatible

Backend:

- free-tier compatible Node host
- architecture must allow self-hosting

Database:

- Supabase free tier or equivalent

WebRTC:

- browser P2P
- free STUN
- TURN adapter optional

Important:

Free tiers change over time. Never hard-code the assumption that a third-party free tier is unlimited.

Create adapters/interfaces for:

- database
- auth
- TURN
- storage
- email
- analytics

This makes providers replaceable.

28. Security

Required:

- HTTPS
- secure cookies
- CSRF protection where applicable
- strict CORS
- CSP
- XSS sanitization
- schema validation
- rate limiting
- room access checks
- permission checks server-side
- message length limits
- file size/type limits
- SSRF prevention
- safe URL parsing
- abuse reporting
- ban system
- session expiry
- brute-force protection

Never trust client role information.

29. Privacy

Default:

- minimum data collection
- guest-first
- no unnecessary tracking
- no recording by default
- camera/mic only after browser permission
- private rooms not discoverable
- ephemeral room state where practical

Provide:

- Privacy Policy
- Terms
- Community/Safety rules

30. Performance

Targets:

- fast landing page
- lazy-load room modules
- dynamic import heavy games
- optimize video/image assets
- minimize WebSocket traffic
- throttle presence updates
- avoid React re-render storms
- virtualize long chats
- use optimistic UI where safe
- handle reconnects gracefully

31. Error States

Must handle:

- invalid room
- expired room
- room full
- wrong password
- camera denied
- microphone denied

- screen-share denied
- WebRTC failure
- socket disconnect
- database unavailable
- invalid media URL
- unsupported media
- autoplay blocked
- participant reconnect
- host leaves
- permission denied

Every error needs a human-readable recovery action.

32. Host Transfer

If owner/host disconnects:

1. detect timeout
2. elect next eligible participant
3. update role
4. broadcast new host
5. preserve room state

Do not immediately destroy a room because one user disconnected.

33. Mobile

Responsive from the beginning.

Mobile requirements:

- touch controls
- bottom sheet chat
- compact participant list
- portrait/landscape support
- camera controls
- screen-share limitations clearly communicated
- PWA install
- safe-area support
- low-bandwidth mode

34. PWA

Include:

- manifest
- icons
- install prompt
- offline landing shell
- app shortcuts where supported
- theme metadata

Realtime room functionality still requires connectivity.

35. Accessibility

Target WCAG-minded implementation:

- keyboard navigation
- visible focus
- semantic buttons
- aria labels
- contrast
- reduced motion
- screen-reader friendly controls
- captions/transcripts only where technically available

36. Premium UI Direction

DARKO must NOT look like a generic Discord clone.

Visual direction:

- cinematic dark interface
- deep layered backgrounds
- soft glass surfaces
- subtle grain
- large expressive typography
- ambient gradients
- responsive micro-interactions

- premium cards
- floating room controls
- elegant transitions
- spatial hierarchy
- strong empty states

Avoid:

- excessive neon
- noisy gradients
- template-dashboard look
- excessive rounded cards
- copying Kosmi layouts

The UI must have its own design language.

37. Design Tokens

Define:

- spacing scale
- typography scale
- radius scale
- shadow/elevation
- surface hierarchy
- motion timing
- z-index layers
- breakpoints

Use CSS variables.

Support dark/light if implemented, but dark mode is the primary visual identity.

38. Admin Dashboard

Pages:

- overview
- active rooms
- reports
- users
- bans

- system health
- feature flags

Metrics:

- active rooms
- active users
- realtime connection count
- WebRTC failure rate
- errors
- reports
- feature usage

Do not store excessive personal analytics.

39. Testing

Unit:

- room permissions
- synchronization calculations
- event validation
- role transitions

Integration:

- room creation/join
- chat
- media sync
- reconnect
- host transfer

E2E:

- guest join
- Google login
- create room
- invite
- watch YouTube
- chat
- voice/video
- screen share
- game

Test desktop + mobile browsers.

40. Development Phases

Phase 1 — Foundation

- Next.js app
- design system
- landing page
- authentication
- guest sessions
- database
- room creation/join

Phase 2 — Realtime

- Socket/WebSocket
- presence
- chat
- roles
- permissions

Phase 3 — WebRTC

- voice
- video
- screen share
- reconnect

Phase 4 — Watch Party

- YouTube
- direct video
- synchronization
- playlist

Phase 5 — Games

- Tic-Tac-Toe
- Connect Four
- Chess
- drawing

Phase 6 — Collaboration

- file P2P
- whiteboard
- link sharing

Phase 7 — Public ecosystem

- public lobby
- moderation
- reports
- admin

Phase 8 — Polish

- animations
- PWA
- accessibility
- performance
- security
- testing
- deployment

41. Acceptance Criteria

The MVP is successful when:

1. A user can create a room without paying.
2. A guest can join through a link without an account.
3. Google login works.
4. Multiple users appear in realtime.
5. Chat works.
6. Voice/video works where WebRTC connectivity permits.
7. Screen sharing works where browser permissions permit.
8. YouTube playback can synchronize.
9. Host controls are enforced server-side.
10. Users can play at least three multiplayer games.
11. Room permissions work.
12. Reconnect does not destroy room state.
13. Mobile layout is usable.
14. PWA can be installed.

15. No paid API is required for the core MVP.
16. The application does not bypass DRM or redistribute protected content.
17. Security validation is implemented before production.

42. MASTER DEVELOPMENT PROMPT

You are a senior staff-level full-stack engineer, realtime systems architect, WebRTC engineer, product designer, security engineer, and QA lead.

Build a production-quality web application called **DARKO**.

DARKO is an original browser-first virtual hangout platform inspired by the general product category of watch-party and virtual-room applications.

The goal is NOT to copy Kosmi's branding, source code, assets, text, or exact UI. Build a clearly original product with superior premium UX.

Core product

Users can:

- create rooms
- join rooms through links
- join as guests
- optionally authenticate with Google
- chat
- voice chat
- video chat
- screen share
- watch supported media together
- synchronize playback
- use playlists
- share links
- transfer files P2P
- use collaborative whiteboard
- play multiplayer games
- customize rooms
- manage roles and permissions
- discover public rooms
- report abuse

Hard constraints

1. Target $\blacksquare$ 0 infrastructure cost.
2. Prefer free/open-source technologies.
3. No paid APIs required for MVP.
4. Do not bypass DRM.
5. Do not proxy protected streaming services.
6. Do not redistribute copyrighted content.
7. Use WebRTC for realtime media.
8. Use a replaceable architecture for TURN/SFU/storage.
9. Guest joining must work.
10. Google login must be optional.
11. UI must be completely original and premium.
12. Desktop, tablet and mobile must work.

Preferred stack

Next.js + TypeScript

Tailwind CSS

Framer Motion

Node.js + TypeScript

Socket.IO/WebSocket

WebRTC

Postgres/Supabase-compatible database

Google OAuth

Zod

Zustand or equivalent

Lucide

PWA

Use alternatives only when there is a strong technical reason.

Engineering rules

- strict TypeScript
- no `any` unless unavoidable and documented
- reusable components
- modular services
- server-side authorization
- typed realtime events
- schema validation
- clean error handling

- no secret keys in client code
- ``.env.example``
- README
- Docker support where useful
- migration scripts
- seed data for development
- unit/integration/e2e tests
- accessible controls
- reduced-motion support

Architecture

Separate:

- UI
- room state
- realtime transport
- media/WebRTC
- authentication
- persistence
- moderation
- games
- media providers

Create provider interfaces so future infrastructure can replace:

- TURN
- database
- storage
- authentication
- analytics
- SFU

Realtime

Design a versioned event protocol.

Every incoming event must:

- be authenticated or associated with a valid guest session
- validate payload
- validate room membership
- validate permission
- be rate limited where necessary

Never trust the client for host/moderator privileges.

WebRTC

Implement:

- offer/answer
- ICE candidate exchange
- STUN configuration
- peer lifecycle
- reconnect
- device changes
- mute/camera states
- screen share
- connection quality

Keep media architecture abstract enough to migrate from P2P mesh to SFU later.

For small rooms, P2P may be acceptable. Do not claim unlimited scalability.

Watch synchronization

Create a room media state.

Use:

- authoritative host/client policy
- timestamp-based synchronization
- drift correction
- explicit play/pause/seek events
- sync request/response

Do not broadcast playback position continuously.

UI

Design DARKO from scratch.

The interface should feel:

- cinematic
- premium
- modern
- immersive
- social

- responsive

Create a consistent design system before building individual pages.

Include:

- landing page
- create room
- join flow
- room
- profile
- settings
- games
- public lobby
- admin

The room must feel like a real virtual space rather than a dashboard.

Room UI

Desktop:

- activity stage
- participant area
- chat
- media controls
- activity switcher
- room controls

Mobile:

- bottom navigation
- bottom sheets
- collapsible panels
- touch-first controls

Games

Build at least:

- Tic-Tac-Toe
- Connect Four
- Chess

Games must have synchronized state and reconnect support.

File sharing

Prefer WebRTC DataChannel for direct P2P transfers.

Implement:

- file metadata
- offer
- accept/reject
- chunk transfer
- progress
- cancel
- completion
- limits

Do not permanently upload files by default.

Security

Implement:

- secure sessions
- CSRF protection where applicable
- CORS
- CSP
- XSS protection
- SSRF protection
- rate limits
- room authorization
- moderation
- abuse reports
- file validation
- URL validation
- role enforcement

Privacy

Collect the minimum necessary data.

Guests should not need personal information.

Do not record voice/video.

Do not expose private rooms in public discovery.

Free deployment

Provide deployment instructions for a free-tier-first setup.

Document:

- frontend deployment
- backend deployment
- database setup
- Google OAuth setup
- environment variables
- WebRTC/STUN
- optional TURN
- PWA
- production security

Do not promise that third-party free tiers are unlimited. Clearly mark quotas and possible future cost points.

Output requirements

When generating the application, work in this order:

1. architecture
2. database schema
3. event protocol
4. design system
5. landing page
6. authentication
7. room lifecycle
8. realtime infrastructure
9. chat/presence
10. WebRTC
11. watch synchronization
12. games
13. file sharing
14. whiteboard
15. public rooms
16. moderation
17. admin
18. PWA
19. tests
20. security audit

21. performance audit

22. deployment documentation

Do not generate fake buttons for core features.

If a feature cannot be implemented using browser capabilities or a free dependency, implement the closest honest alternative and document the limitation.

Before considering the product complete, verify all acceptance criteria in this PRD.

43. Final Product Personality

DARKO should feel like:

"Discord + watch party + virtual room + mini-game lounge + collaborative space"

but should remain an original product with its own visual identity and architecture.

The user should be able to open DARKO, create a room in seconds, send one link, and have a complete virtual hangout without forcing everyone to install software or create accounts.